

CM-204 Capstone Project Proposal

OS Control Room: The MiniKernel Decision Engine

Problem Statement / Objective

Operating systems are usually taught as separate topics: processes, scheduling, memory, files, signals, and concurrency. In a real system, these parts constantly affect each other. A process can be created, wait in a ready queue, run on the CPU, request memory, open a file, block on I/O, wait for a lock, or be terminated by a signal. The objective of this project is to make these hidden decisions visible in one small interactive system.

My project will be a terminal-based MiniKernel simulator called **OS Control Room**. It is not intended to be a real kernel. It is a controlled model of how an operating system manages processes and resources. The user will act like the OS decision maker and will choose what should happen in different situations, for example when a scheduler must pick the next process, when memory is full, when two processes create a deadlock, or when a file write crashes before it is safely completed.

Underlying Approach and Technical Setup / Environment

The project will be implemented as a Linux terminal application, most likely in C or Python. The simulator will manage several artificial processes. Each process will have a PID, a state, remaining CPU time, memory needs, open files, and possible resource requests. The dashboard will show a process table, ready queue, blocked queue, memory table, open file table, lock table, and short diagnostic messages.

The simulation will be interactive rather than only automatic. For example, if P1 holds Lock A and waits for Lock B while P2 holds Lock B and waits for Lock A, the program will detect the deadlock and ask what should be done next. Possible actions could be showing the wait-for graph, killing one process, waiting longer, or applying a lock ordering rule. In a scheduling scenario, the user can choose FIFO, SJF, or Round Robin and compare waiting time, response time, and turnaround time. In a file scenario, the simulator can show why a small log or journal helps after a crash during writing.

Operating Systems Topics Addressed

The project connects several parts of CM-204 in one scenario. It covers processes and process states, CPU scheduling, memory allocation, file descriptors and file access, locks and deadlock, signals, and simple crash recovery. The process table and queues represent the process abstraction. The scheduler part uses FIFO, SJF, Round Robin, and timing metrics. The memory part shows limited memory, allocation decisions, and simple page fault situations. The file part shows opened files, read/write behavior, and durability problems. The concurrency part uses locks, shared resources, wait-for graphs, and deadlock recovery.

Relevant Outcomes / Why This Project Is Interesting

The interesting part of this project is that it does not present OS topics one after another as a theory summary. Instead, it builds one small system where the topics interact. A process may move from ready to running, become blocked on I/O, request a lock, cause a deadlock, or crash during a file operation. The MiniKernel then shows the current state, asks for a decision, and explains the result.

During the presentation, I can create a problem live, let the simulator display the state of the system, make a decision, and explain the consequence using lecture concepts. The final result should feel like a small OS control panel. It shows not only definitions, but also why OS mechanisms behave the way they do and how different parts of the system influence each other.